

UNIT-I

Theory of computation Introduction

Automata theory is the study of abstract computing devices or machines. The machine that do things automatically. This is introduced before computers. Automata theory is now used to understand the basic working mechanism of computers. Here we study about; the mathematical model which do things automatically, decidability (computability) of some languages and complexity classes.

Automata mathematical models

- 1) Finite state machine
- 2) Push down automata
- 3) Turing Machine

computability

- 1) Decidability
- 2) undecidability

Languages

- 1) Regular languages
- 2) context free languages

Complexity classes

1) Time complexity

a) class P

b) class NP

2) space complexity

Some automata concepts used now are

1) Software for designing and checking the behavior of digital circuits.

2) Lexical analyzer - breaking i/p to logical units.

3) Finding the occurrence of certain words in large text.

4) Protocol verification.

Application

1) Nowadays automata theory are used to decide what a computer do at all.

2) what can a computer do efficiently

Basic Mathematical Notation and Techniques:

Types of Proofs

1) Deductive Proof

- step by step reduction
- will have initial statements ~~include~~ hypothesis
- will reduce to conclusion.
- use some accepting principles.

Example 1 →

if $x \geq 4$, then $2^x \geq x^2$.

for $x = 4$ then $16 = 16$

for $x = 5$ then $32 > 25$

Each time RHS (x^2) grows by $\left(\frac{x+1}{x}\right)^2 \rightarrow$

So LHS doubles and RHS grows by 1.5625

ie $2 > 1.5625$

So $2^x \geq x^2$, for $x \geq 4$.

Example 2

If x is the sum of the squares of four positive integers, then $2^x \geq x^2$.

Proof

Let a, b, c, d be 4 +ve integers.

Then $a^2 + b^2 + c^2 + d^2 \geq 4$

if $x = a^2 + b^2 + c^2 + d^2$

so $x \geq 4$

Hence $2^x \geq x^2$

2) Proving If - Then statements:

If (something) then (something else)

if part $\rightarrow$ hypothesis

then part $\rightarrow$ conclusion

3) Proving if and only if:

two ways

$\rightarrow$ proving two if and only if.

4) Proving the contrapositive:

so prove if H then C by if not

$\&$ then not H . (if $a > b$ then a greater b)

5) Proof by contradiction:

if H and not C then something else.

Proof by counterexamples:

→ If we have to prove some stmt is not true assign some values such that stmt become false.

7) Inductive Proof (Mathematical Induction)

To prove some statement $S(n)$ we have two things.

1) Basic : for minimal i value we have to show $S(i)$ is true.

2) Induction : Assuming statement is true for $S(n)$ we have to prove for $S(n+1)$.

Example

Prove
$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

(4 mark)
✗

Basic

for $i=1$, (LHS)

$$\sum_{i=1}^1 i^2 = 1^2 = 1$$

(RHS)

$$\begin{aligned} \frac{n(n+1)(2n+1)}{6} &= \frac{1(2)(2+1)}{6} \\ &= \frac{2 \times 3}{6} \\ &= 1 \end{aligned}$$

Induction

Assume

for all n ,

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

apply

$$n = n+1$$

$$\sum_{i=1}^{n+1} i^2 + (n+1)^2 = \frac{(n+1)(n+1+1)(2(n+1)+1)}{6}$$

$$= \frac{(n+1)(n+2)(2n+2+1)}{6}$$

$$= \frac{(n+1)(n+2)(2n+3)}{6}$$

$$= \frac{2n(n^2 + n + 2n + 1)(2n+3)}{6}$$

$$= \frac{(n^2 + 3n + 1)(2n+3)}{6}$$

$$= \frac{2n^3 + 3n^2 + n}{6}$$

$$= \frac{2n^3 + 3n^2 + 6n^2 + 9n + 2n + 3}{6}$$

$$= \frac{2n^3 + 9n^2 + 11n + 3}{6}$$

$$\frac{n(n+1)(2n+1)}{6} + \frac{(n+1)^2}{1}$$

$$\frac{n(n+1)(2n+1) + 6(n^2 + 2n + 1)}{6} :$$

$$\frac{2n^3 + 9n^2 + 13n + 3}{6} = \frac{2n^3 + 9n^2 + 13n + 3}{6}$$

is true for $n+1$. Hence by induction it is proved that

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

General concepts of Automata theory:

1) Alphabets : finite, non-empty set of symbols

symbol : Σ

eg: $\Sigma = \{0, 1\}$, the binary alphabet.

2) String : finite sequence of symbols

Chosen from some alphabet.

eg: 01001 with $\Sigma = \{0, 1\}$

3) Empty string:

String with zero occurrence of symbol in language.

Symbol: ϵ .

4) length of a string:

Number of positions for symbols in the string. Length of ababa is 5.

Symbol: $|ababa| = 5$.

if $w = ababa$ then $|w| = 5$.

5) power of Alphabet:

Set of all strings of a certain length from that alphabet by using an exponential notation.

if $\Sigma = \{a, b\}$ then $\Sigma^0 = \{\epsilon\}$

then $\Sigma^1 = \{a, b\}$

$\Sigma^2 = \{aa, ab, ba, bb\}$

Positive closure:

Denoted as $\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \dots$
(one or more occurrence)

Kleene closure

$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$

(0 or more occurrence)

SRM

Language:

A set of strings all of which are chosen from some Σ^* , is called a language.

If Σ is an alphabet, and $L \subseteq \Sigma^*$ then L is a language over Σ .

Example

The language of all strings over alphabet $\Sigma = \{a, b\}$ which ends with ba .

Extra

~~Defective~~ proof (set example)
contradiction

Prob

Let S be a finite subset of some infinite set U . Let T be the complement of S with respect to U . Then T is infinite.

Proof

ste 1) S is finite $\Rightarrow ||S|| = n$
 $n = \text{the integer}$

2) U is infinite $\Rightarrow ||S|| = p$
 $p = \text{not integer.}$

3) T complement of S

$$\text{ie } S \cup T = U \quad \text{and} \quad S \cap T = \emptyset$$

assume that T is finite so

$$|T| = m \quad \text{some int integer.}$$

$$|U| = |S| + |T|$$

$$= n + m \quad (\text{both } n \text{ and } m)$$

Then $|U|$ should also be a positive integer that means finite. but we know that U is infinite. hence T is also infinite.

Finite Automata :

Finite automata is a mathematical model which has a set of states and its transition from one state to another state based on the given input. Finite state machine does not have any memory so its transition exactly depends on current state and current input symbol. There are two types of finite automata

1) Deterministic finite automata (DFA)

2) Non-deterministic finite automata (NFA)

DFA : It has transition for a single input to only one state. For all inputs there is a transition from one state to exactly one state.

NFA : On a single input it may or may not have a transition and also it may have any number of transitions, on a single input from one state.

Simple example

Automatic door $\rightarrow$ has two states open and close. The external input that can be

given to the system is if some one
come near the door Let we take that as 1
and if nobody is here we take that as 0.

1 $\rightarrow$ some one is standing near the door

0 $\rightarrow$ nobody is here near the door.

So the transition diagram will be



Here we don't have any starting
or final state. This is a simple
examples for FSM or finite automata.

~~Example~~

Technique:

to express a finite state machine
we use

1) Transition Diagram

2) Transition Table

3) Type notation

Transition Diagram:

It is a directed graph with nodes denoting states and labeled arc denoting transition from one state to another. The label denote the ip symbol. The starting state is denoted as



and the final state with double circle as,



Transition Table

	ip symbols
states	transition

eg:

	0	1
open	close	open
close	close	open.

tuple notation

This depends on either NFA or

DFA.

Tuple notation for DFA :

$$(Q, \Sigma, \delta, q_0, F)$$

Q = set of states

Σ = i/p symbols

δ = transition function, which is like

$$\delta(q, a) = p$$

$q \rightarrow$ current state

$a \rightarrow$ current i/p symbol

$p \rightarrow$ next state

q_0 = starting state

F = final state.

Tuple notation for NFA:

Same as DFA but F is not a single state it is a set of states.

Transition function,

$$\delta(q, a) = \{p, r\}$$

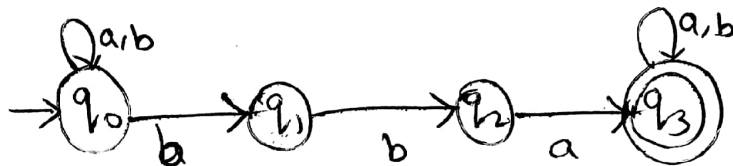
all possible next states.

Formal Examples:

1) Design an NFA and DFA for a language of strings over the alphabet $\Sigma = \{a, b\}$ the strings which contain aba as substring.

Solution (NFA)

Transition diagram



Table

	a	b
q ₀	q ₀	{q ₀ , q ₁ }
q ₁	-	q ₂
q ₂	q ₃	-
q ₃	q ₃	q ₃

5-tuple notation $(Q, \Sigma, \delta, q_0, F)$

$Q = \{q_0, q_1, q_2, q_3\}$

$\Sigma = \{a, b\}$

$q_0 = q_0 \quad F = \{q_3\}$

$\delta(q_0, a) = q_0$

$\delta(q_0, b) = \{q_0, q_1\}$

$\delta(q_1, b) = q_2$

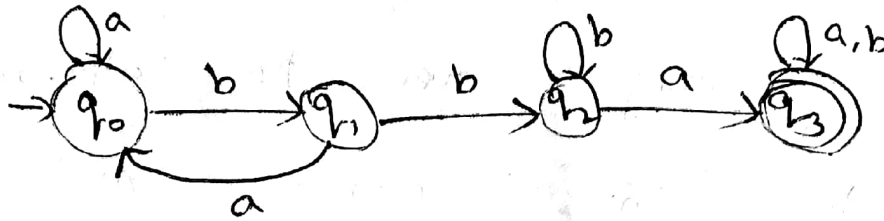
$\delta(q_2, a) = q_3$

$\delta(q_3, a) = q_3$

$\delta(q_3, b) = q_3$

SRM

DFA (Transition Diagram)



Transition table

	a	b
q ₀	q ₀	q ₁
q ₁	q ₀	q ₂
q ₂	q ₃	q ₂
q ₃	q ₃	q ₃

Tuple notation

$$\{Q, \Sigma, \delta, q_0, F\}$$

$$Q = \{q_0, q_1, q_2, q_3\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = q_0$$

$$F = q_3$$

$$\delta(q_0, a) = q_0$$

$$\delta(q_0, b) = q_1$$

$$\delta(q_1, a) = q_0$$

$$\delta(q_1, b) = q_2$$

$$\delta(q_2, a) = q_3$$

$$\delta(q_2, b) = q_2$$

$$\delta(q_3, a) = q_3$$

$$\delta(q_3, b) = q_3$$

Example

Extended Transition Diagram:

Transition function δ describes the transition on single input from a single state. The extended transition function describes what happens when we start in any state and follow any sequence of inputs.

$\delta \rightarrow$ Transition function

$\hat{\delta} \rightarrow$ Extended Transition function.

Basic

$$\hat{\delta}(q, \epsilon) = q.$$

Induction

if i/p $w = \alpha a$ over $\Sigma = \{a, b\}$

$\alpha \in \Sigma^+$ then,

$$\hat{\delta}(q, w) = \delta(\hat{\delta}(q, \alpha), a)$$

Example

for the same example in ⑧ if

i/p = abba then,

$$\begin{aligned}\hat{\delta}(q_0, abba) &= \delta(\hat{\delta}(q_0, abb), a) \\ &= \delta(\delta(\hat{\delta}(q_0, ab), b), a)\end{aligned}$$

$$= \delta(\delta(\delta(\hat{\delta}(q_0, a), b), b), a)$$

$$= \delta(\delta(\delta(\delta(\hat{\delta}(q_0, \epsilon), a), b), b), a)$$

$$= \delta(\delta(\delta(\delta(q_0, a), b), b), a)$$

$$= \delta(\delta(\delta(q_0, b), b), a)$$

$$= \delta(\delta(q_1, b), a)$$

$$= \delta(q_2, a)$$

$$= q_3 \text{ (final state)}$$

The language accepted by DFA:

A language of DFA can be defined as, $A = (Q, \Sigma, \delta, q_0, F)$ - DFA

$$L(A) = \{w \mid \hat{\delta}(q_0, w) \text{ is in } F\}$$

$L(A) \rightarrow$ Language accepted by A (DFA)

That is, the language of A is the set of strings w that take the start state q_0 to one of the accepting states. If L is $L(A)$ for some DFA A, then we say L is a regular language.

Regular Language

when a language is accepted by some DFA or NFA then it is called Regular Language.

NFA

Extended Transition function:

The extended transition function for NFA we start with basic and in induction we have many possible solutions. It is defined as,

Basic

$$\hat{\delta}(q, \epsilon) = q$$

Induction

if $w = \alpha a$ a is the final symbol of w and α is the rest of w , and

$$\hat{\delta}(q, \alpha) = \{p_1, p_2, p_3 \dots p_k\} \text{ and}$$

$$\bigcup_{i=1}^k \delta(p_i, a) = \{r_1, r_2 \dots r_m\}$$

Then, $\hat{\delta}(q, w) = \{r_1, r_2 \dots r_m\}$

Example

For the same example in 8 is if
i/p is abbbba then

$$\hat{\delta}(q_0, \epsilon) = q_0$$

$$\hat{\delta}(q_0, a) = \delta(q_0, a) = q_0$$

$$\hat{\delta}(q_0, ab) = \delta(q_0, b) = \{q_0, q_1\}$$

$$\hat{\delta}(q_0, abb) = \delta(q_0, b) \cup \delta(q_1, b) = \{q_1, q_2\}$$

$$\hat{\delta}(q_0, abbb) = \delta(q_1, b) \cup \delta(q_2, b) = \{q_2\}$$

$$\hat{\delta}(q_0, abbbba) = \delta(q_2, a) = q_3$$

on i/p abbbba the NFA remains in
 q_3 i.e. accepting state. Hence the given
i/p is accepted by NFA.

The Language of NFA

if $A = (Q, \Sigma, \delta, q_0, F)$ is an
NFA, then,

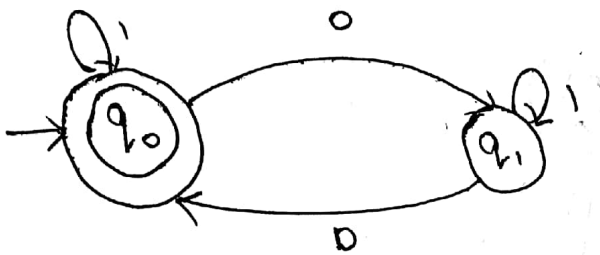
$$L(A) = \{w \mid \hat{\delta}(q_0, w) \cap F \neq \emptyset\}$$

that is, $L(A)$ is the set of strings w in Σ^* such that
 $\hat{\delta}(q_0, w)$ contains at least one accepting state.

Problem set

1) Design a DFA to accept the language

$$L = \{w \mid w \text{ has even number of 0's over } \Sigma = \{0, 1\}\}$$



For all one it remain in same state and odd number of 0's it goes to q_1 state, even number of 0's come back to q_0 state which is final state.

Transition table

	0	1
$\rightarrow q_0$	q_1	q_0
q_1	q_0	q_1

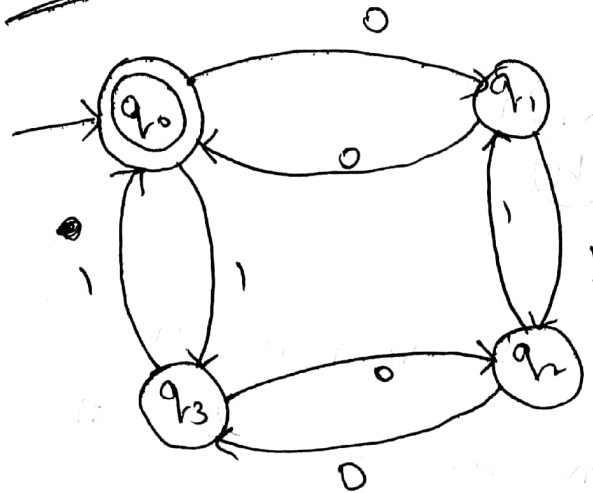
Tuple notation

$$A = \{[q_0, q_1], \{0, 1\}, \delta, q_0, q_0\}$$

2) Design a DFA to accept the language

$L = \{w \mid w \text{ has both an even number of 0's and an even number of 1's}\}$

Transition Diagram



Transition Table

	0	1
q_0	q_1	q_3
q_1	q_0	q_2
q_2	q_3	q_1
q_3	q_2	q_0

Tuple notation

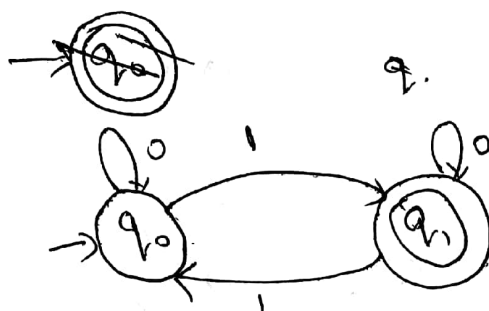
$$A = (\{q_0, q_1, q_2, q_3\}, \{0, 1\}, \delta, q_0, q_0)$$

Design a DFA to accept the language

$L = \{w \mid w \text{ has an odd number of 1's}$

over $\Sigma = \{0, 1\}$

Transition Diagram



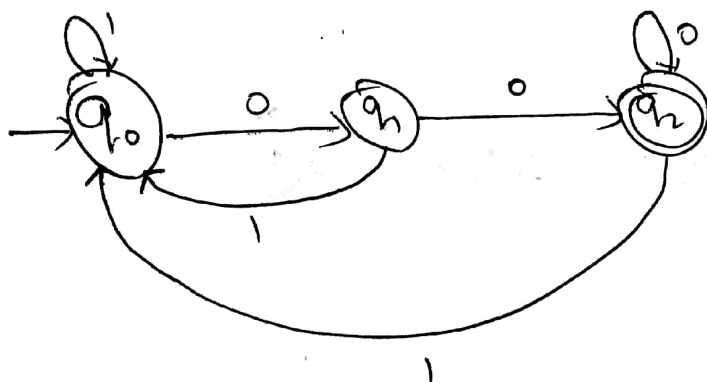
Tuple notation

$$A = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, q_1)$$

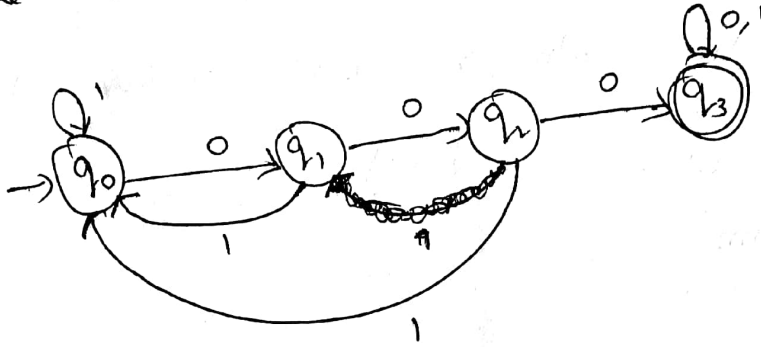
4) Design a DFA to accept the language

over the alphabet $\Sigma = \{0, 1\}$

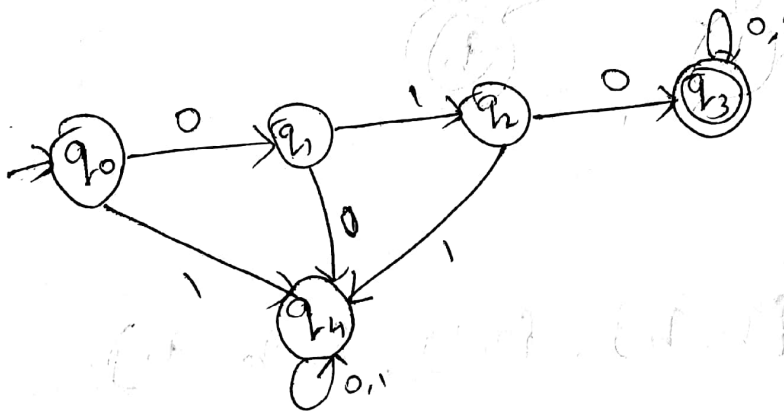
a) Set of strings ending with 00.



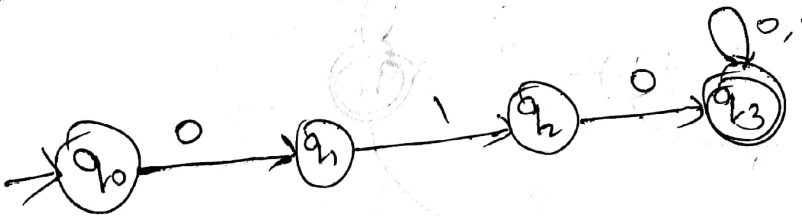
b) The set of all strings with 3 consecutive



c) The set of strings starts with 010



5. design a DFA that accepts the language over the alphabet $\{0,1\}$, the set of strings starts with 010.



Equivalence of Deterministic and Non-Deterministic Finite automata:

Equivalence $\rightarrow$ we have to prove that both NFA and DFA accepts same kind of languages. we can convert NFA to DFA using subset construction method.

Def The subset construction starts from an NFA, $N = (Q_N, \Sigma, \delta_N, q_0, F_N)$. we have to construct DFA, $D = (Q_D, \Sigma, \delta_D, q_0, F_D)$, such that $L(D) = L(N)$

$\rightarrow$ i/p is common

$\rightarrow$ both have same starting state

the other components can be constructed as,

* Q_D is subset of Q_N .

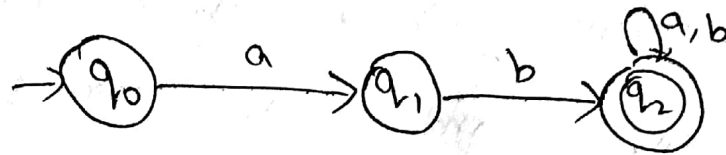
* F_D is the set of subsets S of Q_N such that $S \cap F_N \neq \emptyset$. That is F_D is all sets of N 's states that include at least one accepting state of N .

* for each set $S \subseteq Q_N$ and for each

i/p symbol a in Σ , $\delta_D(S, a) = \bigcup_{p \in S} \delta_N(p, a)$

Example

consider the NFA,



accepts language over $\Sigma = \{a, b\}$ accepts string starts with a, b.

The transition table is,

	a	b
$\rightarrow q_0$	q_1	ϕ
q_1	ϕ	q_2
q_2^*	q_2	q_2

$$Q_N = \{q_0, q_1, q_2\}$$

all possible subsets are

$$\text{subset } Q_N = \{\phi, q_0, q_1, q_2, \{q_0, q_1\}, \{q_0, q_2\}, \{q_1, q_2\}, \{q_0, q_1, q_2\}\}$$

Construct a Transition table with $i) P \subseteq \{0,1\}$ and all subsets at one side

	0	1
ϕ	ϕ	ϕ
$\rightarrow q_0$	q_1	ϕ
q_1	ϕ	q_2
q_2^*	q_2	q_2
$\{q_0, q_1\}$	q_1	q_2
$\{q_0, q_2\}^*$	$\{q_1, q_2\}$	q_2
$\{q_1, q_2\}^*$	q_2	q_2
$\{q_0, q_1, q_2\}^*$	$\{q_1, q_2\}$	q_2

8 or 16 marks

Now rename all subsets,

$\{\phi, q_0, q_1, q_2, \{q_0, q_1\}, \{q_0, q_2\}, \{q_1, q_2\}, \{q_0, q_1, q_2\}\}$

as, $\{A, B, C, D^*, E, F^*, G^*, H^*\}$

Denote starting state as $\rightarrow B$.

and put a star over all subsets having final state q_2 .

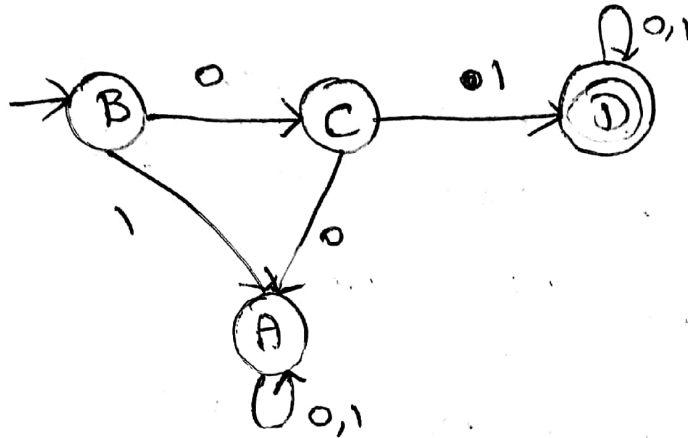
Redraw the transition table.

	0	1
A	A	A
→ B	C	A
C	A	D
D*	D	D
E	B	D
F*	G	D
G*	D	D
H*	A	D

Now remove all unreachable nodes from starting state.

	0	1
→ B	C	A
A	A	A
C	A	D
D*	D	D

Transition Diagram



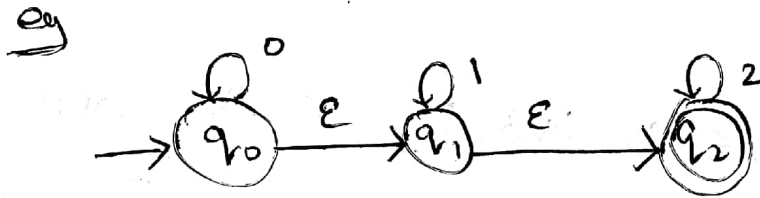
Theorems

- 1) If $D = (Q_D, \Sigma, \delta_D, \{q_0\}, F_D)$ is the DFA constructed from NFA $N = (Q_N, \Sigma, \delta_N, \{q_0\}, F_N)$ by the subset construction, then $L(D) = L(N)$.
- 2) A Language L is accepted by some DFA if and only if L is accepted by some NFA.

Finite Automata with Epsilon - Transition

It is possible in NFA that NFA is allowed to make transition spontaneously without receiving an input symbol. This move is called ϵ -move.

ϵ - any number of times.



Formal Notation

$$A = (Q, \Sigma, \delta, q_0, F)$$

$Q, \Sigma, q_0, F \Rightarrow$ same as NFA.

δ has arguments,

1) A state in Q

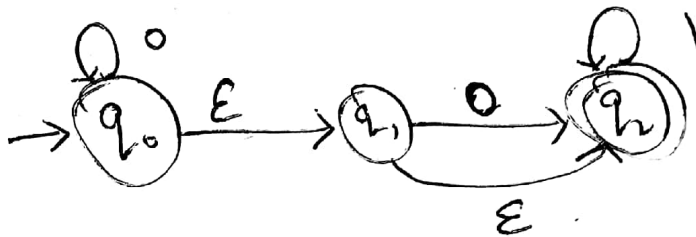
2) A new input symbol ϵ is added

to the input symbols.

Epsilon Closure :

ϵ -closure of a state q is finding every state that can be reached from q along any path whose arcs are all labelled ϵ .

eg



$$\epsilon\text{-closure}(q_0) = \{q_0, q_1, q_2\}$$

$$\epsilon\text{-closure}(q_1) = \{q_1, q_2\}$$

$$\epsilon\text{-closure}(q_2) = \{q_2\}$$

Regular Expressions and Languages

A language accepted by finite automata is called regular language. It can be easily described by simple expressions called regular expressions.

operations of regular expressions

1) Union

The union of two languages L_1 and L_2 is $L_1 \cup L_2$, the set of strings that are in either L_1 or L_2 or both. ~~$L_1 \cup L_2 = \{ \text{any string in } L_1 \text{ or } L_2 \}$~~

eg

if $L = \{001, 10, 111\}$ and

$M = \{\epsilon, 001\}$

then

$L \cup M = \{\epsilon, 001, 10, 111\}$

2) concatenation

The concatenation of two language L_1 and L_2 is formed by choosing a string in L_1 following it by a string in L_2 in all possible combinations.

It is denoted as $L_1.L_2$.

eg if $L = \{001, 10, 11\}$ and $m = \{\epsilon, 001\}$

then $LM = \{001, 10, 11, 001001, 10001, 111001\}$

3) Closure (Kleen closure)

The Kleen closure of a language L is denoted L^* is defined as the set of strings that can be formed by taking any number of strings from L , possibly with repetition.

eg if $L = \{10, 1\}$

then $L^* = \{\epsilon, 10, 1, 101, 1010, 110, \dots\}$

Construction of Regular Expressions:

Let Σ be an alphabet. The regular expressions over Σ are the sets that they denoted are defined as follows.

Basic 1) ϕ is the regular expression denoting the language $L(\phi) = \phi$.

2) ϵ is the regular expression denoting the language $L(\epsilon) = \epsilon$.

3) If e and f are regular expressions then

$$E \cup F = e + f \quad (\text{union})$$

$$E \cdot F = e \cdot f \quad (\text{concatenation})$$

$$\underline{E^*} = e^* \quad (\text{Kleene closure})$$

$E \cup F \rightarrow$ regular languages.

Example

i) $\epsilon \rightarrow$ (empty string)

ii) $L(0^*) = \{\epsilon, 0, 00, 000, \dots\}$



iii) 0^* - string starts with 0 followed by zero or more 0's.

iv) $0+1$ - either 0 or 1.

v) $(0+1)^*$ - any number of 0's and 1's.

Precedence of Regular expression operators:

i) $*$ - highest precedence.

ii) $\cdot$ - next

iii) $+$ - last.

Example

1) Set of strings over the alphabet $\Sigma = \{a, b\}$ with ab as substring.

$$(a+b)^* ab (a+b)^*$$

2) The set of strings of a 's and b 's starting with a and ending with ab .

$$a (a+b)^* ab.$$

3) The set of strings of 0 's and 1 's whose number of 0 's is divisible by five.

$$(00000 + 1)^*$$

4) $\{a^n : n \text{ is divisible by } 2 \text{ or } 3\}$

$$(aa)^* + (aaa)^*$$

5) the set of all strings over $\{0,1\}$ which has almost two zeros.

$$1^* 0 1^* 0 1^*$$

English description of language

1) $a^* + (ab+ba)^*$

language of string over $\Sigma = \{a,b\}$ contain

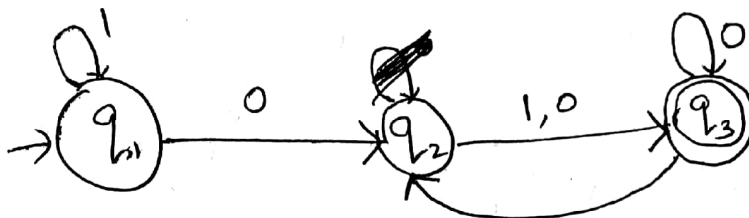
Set of all strings either only a's

or if a b occur it should be along with a. i.e. No two or more b's occur consecutively.

FA to RE

$$R_{ij}^{(k)} = R_{ij}^{(k-1)} + R_{ik}^{(k-1)} [R_{kk}^{(k-1)}]^* R_{kj}^{(k-1)}$$

DFA



$$R_{11}^{(0)} = \epsilon + 1$$

$$R_{12}^{(0)} = 0$$

$$R_{13}^{(0)} = \phi$$

$$R_{21}^{(0)} = \phi$$

$$R_{22}^{(0)} = \epsilon$$

$$R_{23}^{(0)} = 0 + 1$$

$$R_{31}^{(0)} = \phi$$

$$R_{32}^{(0)} = 1$$

$$R_{33}^{(0)} = 0 + \epsilon$$

$$R_{11}^{(1)} = 1^*$$

$$R_{12}^{(1)} = 1^* 0$$

$$R_{13}^{(1)} = \phi$$

$$R_{21}^{(1)} = \phi$$

$$R_{22}^{(1)} = \epsilon$$

$$R_{23}^{(1)} = 0 + 1$$

$$R_{31}^{(1)} = \phi$$

$$R_{32}^{(1)} = 1$$

$$R_{33}^{(1)} = \epsilon + 0$$

$$R_{11}^{(2)} = 1^*$$

$$R_{12}^{(2)} = 1^* 0$$

$$R_{13}^{(2)} = 1^* 0 (0 + 1)$$

$$R_{21}^{(2)} = \phi$$

$$R_{22}^{(2)} = \epsilon$$

$$R_{23}^{(2)} = 0 + 1$$

$$R_{31}^{(2)} = \phi$$

$$R_{32}^{(2)} = 1$$

$$R_{33}^{(2)} = (\epsilon + 0 + 1(0 + 1))$$

$$R_{11}^{(1)} = R_{11}^{(0)} + R_{11}^{(0)} [R_{11}^{(0)}]^* R_{11}^{(0)}$$

$$= (\epsilon + 1) + (\epsilon + 1) (\epsilon + 1)^* (\epsilon + 1)$$

$$= (\epsilon + 1) + 1^*$$

$$= 1^*$$

$$\begin{aligned}
 & i=1 \quad j=2 \quad k=1 \\
 R_{12}^{(1)} &= R_{12}^{(0)} + R_{11}^{(0)} [R_{11}^{(0)}]^* R_{12}^{(0)} \\
 &= 0 + (\varepsilon+1) (\varepsilon+1)^* 0
 \end{aligned}$$

$$= 0 + 1^* 0 = 1^* 0$$

$$\begin{aligned}
 R_{13}^{(1)} &= R_{13}^{(0)} + R_{11}^{(0)} [R_{11}^{(0)}]^* R_{13}^{(0)} \\
 &= \phi + (\varepsilon+1) (\varepsilon+1)^* \phi
 \end{aligned}$$

$$= \phi$$

$$\begin{aligned}
 R_{21}^{(1)} &= R_{21}^{(0)} + R_{21}^{(0)} [R_{11}^{(0)}]^* R_{11}^{(0)} \\
 &= \phi + \phi (\varepsilon+1)^* (\varepsilon+1)
 \end{aligned}$$

$$= \phi$$

$$\begin{aligned}
 R_{22}^{(1)} &= R_{22}^{(0)} + R_{21}^{(0)} [R_{11}^{(0)}]^* R_{12}^{(0)} \\
 &= \varepsilon + \phi (\varepsilon+1)^* 0
 \end{aligned}$$

$$= \varepsilon$$

$$\begin{aligned}
 R_{23}^{(1)} &= R_{23}^{(0)} + R_{21}^{(0)} [R_{11}^{(0)}]^* R_{13}^{(0)} \\
 &= (0+1) + \phi (\varepsilon+1)^* \phi \\
 &= (0+1)
 \end{aligned}$$

$$\begin{aligned}
 R_{31}^{(1)} &= R_{31}^{(0)} + R_{31}^{(0)} [R_{11}^{(0)}]^* R_{11} \\
 &= \phi + \phi [\varepsilon + 1]^* (\varepsilon + 1) \\
 &= \phi
 \end{aligned}$$

$$\begin{aligned}
 R_{32}^{(1)} &= R_{32}^{(0)} + R_{31}^{(0)} \left[\frac{R_{KK}^{(0)}}{R_{11}} \right]^* R_{12} \\
 &= [1 + \phi (\varepsilon + 1)^*] 0 \\
 &= 1
 \end{aligned}$$

$$\begin{aligned}
 R_{33}^{(1)} &= R_{33}^{(0)} + R_{31}^{(0)} [R_{11}^{(0)}]^* R_{13} \\
 &= (\varepsilon + 0) + (\phi) (\varepsilon + 1)^* \phi \\
 &= \varepsilon + 0
 \end{aligned}$$

$$\begin{aligned}
 R_{11}^{(2)} &= R_{11}^{(1)} + R_{12}^{(1)} [R_{22}^{(1)}]^* R_{21} \\
 &= \cancel{(\varepsilon + 1)} + 1^* + 1^* 0 (\varepsilon)^* \phi \\
 &= 1^*
 \end{aligned}$$

$$\begin{aligned}
 R_{12}^{(2)} &= R_{12}^{(1)} + R_{12}^{(1)} [R_{22}^{(1)}]^* R_{22} \\
 &= 1^* 0 + 1^* 0 [\varepsilon^*] \varepsilon \\
 &= 1^* 0
 \end{aligned}$$

$$R_{13}^{(2)} = R_{13}^{(1)} + R_{12}^{(1)} [R_{22}^{(1)}]^* R_{23}$$

$$= \phi + 1^* 0 (E)^* (0+1)$$

$$= 1^* 0 (0+1)$$

$$R_{21}^{(2)} = R_{21}^{(1)} + R_{22}^{(1)} [R_{22}^{(1)}]^* R_{21}$$

$$= \phi + (E) (E)^* \phi$$

$$= \phi$$

$$R_{22}^{(2)} = R_{22}^{(1)} + R_{22}^{(1)} (R_{22}^{(1)})^* R_{22}$$

$$\neq \phi \neq \phi = E + E (E)^* E$$

$$\neq \phi$$

$$= E$$

$$R_{23}^{(2)} = R_{23}^{(1)} + R_{22}^{(1)} (R_{22}^{(1)})^* R_{23}$$

$$= (0+1) + E (E)^* (0+1)$$

$$= (0+1)$$

$$R_{31}^{(2)} = R_{31}^{(1)} + R_{32}^{(1)} (R_{22}^{(1)})^* R_{21}$$

$$= \phi + 1 (E)^* \phi$$

$$= \phi$$

$$\begin{aligned}
 R_{32}^{(2)} &= R_{32}^{(1)} + R_{32}^{(1)} (R_{22}^{(0)})^* R_{22} \\
 &= 1 + 1 (\epsilon)^* \epsilon \\
 &= 1
 \end{aligned}$$

$$\begin{aligned}
 R_{33}^{(2)} &= R_{33}^{(1)} + R_{32}^{(1)} (R_{22}^{(0)})^* R_{23} \\
 &= (\epsilon + 0) + 1 (\epsilon)^* (0 + 1) \\
 &= (\epsilon + 0) + 1 (0 + 1) \\
 &= \cancel{1 (0 + 1)} = 0 (\epsilon + 0 + 1 (0 + 1))
 \end{aligned}$$

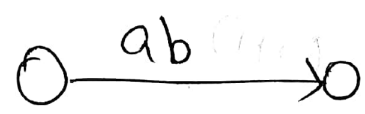
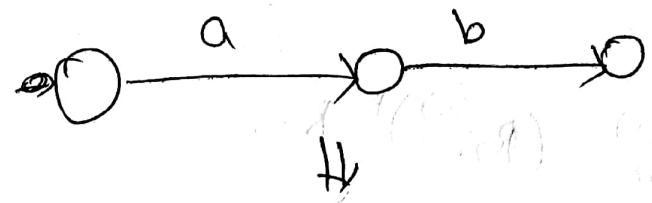
final equation

$$\begin{aligned}
 R_{13}^{(3)} &= R_{13}^{(2)} + R_{13}^{(2)} [R_{33}^{(2)}]^* R_{33} \\
 &= 1^* 0 (0 + 1) + 1^* 0 (0 + 1) [\epsilon + 0 + 1 (0 + 1)]^* (\epsilon + 0 + 1 (0 + 1)) \\
 &= 1^* 0 (0 + 1) + 1^* 0 (0 + 1) [0 + 1 (0 + 1)]^* \\
 &= 1^* 0 (0 + 1) [0 + 1 (0 + 1)]^*
 \end{aligned}$$

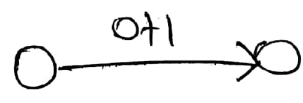
DFA $\rightarrow$ RE [state elimination method]

$\rightarrow$ eliminate all nodes other than final node.

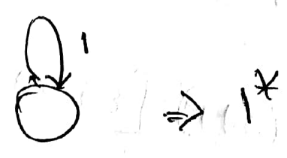
sequential edges



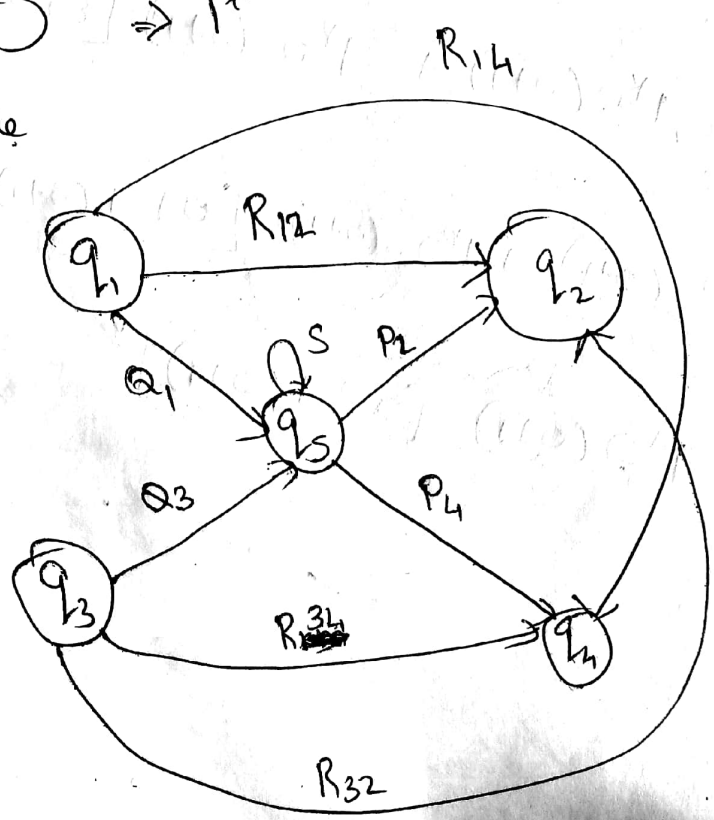
Parallel edges



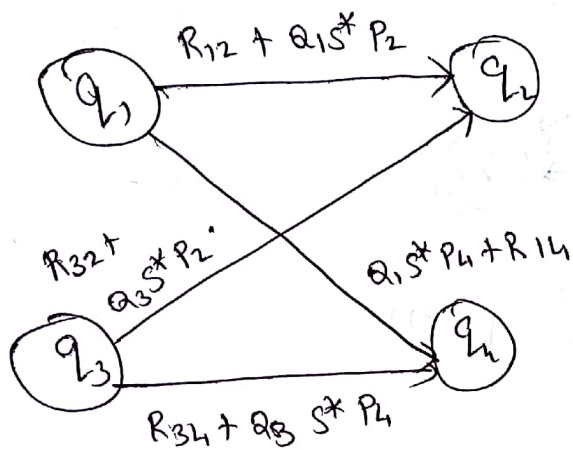
Self loop



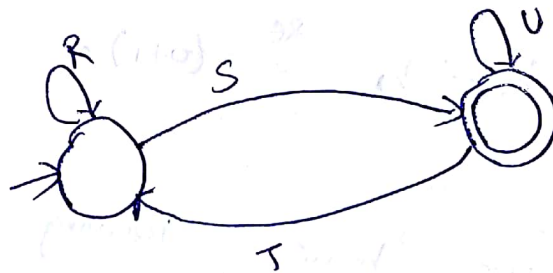
intermediate node



after removing

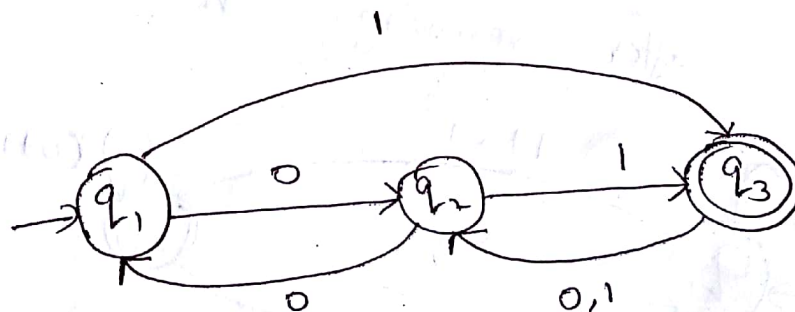


finally when we have starting and final states.

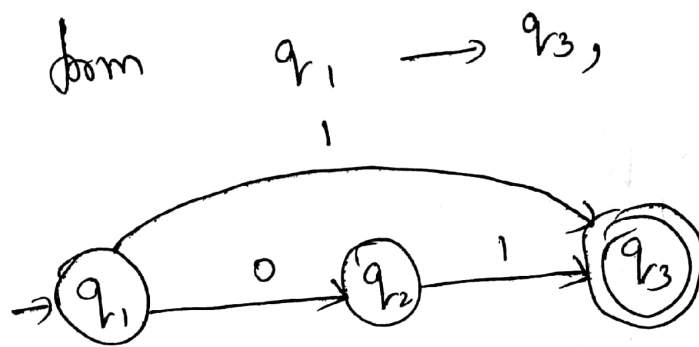


$$RE \Rightarrow (R + SU^*T)^*SU^*$$

Example



do remove q_2 we have to find the path from q_1 to q_3 , q_3 to q_1 , q_1 to q_1 and q_3 to q_3 .



so form $q_1 - q_3 \xRightarrow{RE} 1 + 01$

form $q_3 \rightarrow q_1$,

we don't have any direct path. so

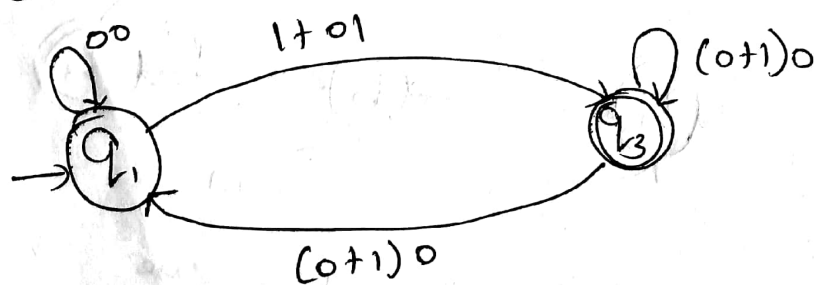
$q_3 - q_1 \xRightarrow{RE} (0+1)0$

similarly, we have looping structure to

q_1 through q_2 with i/p 00. and

form q_3 through q_2 with i/p $(0+1)0$.

finally after removing q_2 the DFA will be



So finally the regular expression for the given DFA will be,

$$\Rightarrow (00 + (1+01)(0+1)0)^* (1+01)((0+1)0)^*$$

$$(e+1)^* = 1^*$$

$$1^*(e+1) = 1^*$$

$$(e+1)1^* = 1^*$$

$$(e+1) + 1^* = 1^*$$

$$0 + 1^*0 = 1^*0$$

$$0 + 01^* = 01^*$$

$$\varepsilon + 00^* = 0^*$$

$$\phi 0 = \phi$$

$$\phi + 0 = 0$$

$$\phi^* = \varepsilon$$

$$11^* \neq 1^*$$

$$1^*1 \neq 1^*$$

$$(01)^* \neq 0^*1^*$$

$$\varepsilon 0 = 0$$